

User management API and Swagger-Based Security Testing

Seminar Report for COMP.SEC.300 Secure Programming

Author: Nguyen Tri Tin

Course context: Secure Programming

Technology stack: Fastify, TypeScript, TypeORM, MariaDB, JWT, bcrypt, Swagger

Date: May 2026

Focus: Demostate secure programming principles through implemented controls, testable API behavior, and evidence-based Swagger demonstrations

Abstract

The design, implementation, and security assessment of a basic CRUD user administration API are presented in this study. Using Fastify, TypeScript, TypeORM, MariaDB, bcrypt password hashing, JSON Web Tokens, signed HttpOnly cookies, and Swagger documentation, the project was constructed as a brief secure programming case study. In addition to implementing fundamental user management, the work's primary goal is to show how popular secure programming concepts may be used and evaluated in a real-world backend application.

Endpoints for user creation, login, logout, and secure user retrieval are all included in the API. The work focuses on password protection, secure response processing, input validation, authentication, and manual security testing using Swagger. Valid user creation, incorrect email formats, weak passwords, payloads that seem like SQL injection or XSS, unauthenticated access, and logout behavior are all covered in several test cases. The outcome is a brief but comprehensive example of how a backend may prevent the exposure of private fields like passwords or password hashes, reject dangerous input, and safeguard sensitive routes.

1.Introduction

Web APIs often store data in a database and handle user-controlled input. Because of this, attacks like SQL injection, cross-site scripting, poor authentication, credential theft, and excessive data exposure find them appealing targets. If input is not verified, passwords are stored improperly, authentication is not enforced, or database entities are provided straight to clients, even a basic user administration API may have significant problems.

In order to manage people, this project develops a modest CRUD API with authentication. In order to focus primarily on secure programming techniques and their straightforward demonstration during a seminar session, the API is purposefully kept small. The evaluator may repeat the primary security test cases straight from the browser using Swagger, which serves as both documentation and a manual testing interface.

2. Research Question and Objectives

The central research question of this work is:

How can a small CRUD API for user management be designed and tested so that it demonstrates practical secure programming principles?

To answer this question, the project focuses on the following objectives:

- Implement a basic user management API with secure authentication.
- Validate user input before it is processed or stored.
- Protect sensitive routes using cookie-based JWT authentication.
- Hash passwords instead of storing plaintext passwords.

- Return only safe user objects to prevent leakage of password hashes.
- Use Swagger/OpenAPI to document endpoints and demonstrate manual security tests.
- Evaluate the behavior of the API using positive and negative test cases.

3. Methodology

A realistic build-and-test approach is used in this study. First, TypeScript and Fastify were used to create a backend API. Second, the most crucial components of the application—input validation, password management, authentication, and response filtering—now have security controls. Third, in order to test and explicitly explain the API behavior during the presentation, Swagger schemas were created for the endpoints. Lastly, manual test cases created to simulate both intended and malicious inputs were used to assess the implementation.

Phase	Activity	Purpose
Design	Define endpoints for user creation, login, logout, and protected user retrieval.	Clarify the attack surface and data flow.
Implementation	Build the API using Fastify, TypeScript, TypeORM, MariaDB, bcrypt, JWT, and cookies.	Create a working backend that can be tested.
Security controls	Add validation, password hashing, signed HttpOnly cookie authentication, and safe response mapping.	Apply secure programming principles.
Testing	Use Swagger to send valid and invalid requests.	Demonstrate expected security behavior.
Evaluation	Check HTTP status codes, response bodies, and database state.	Verify whether controls work as intended.

4. Background and Secure Programming Principles

4.1 Input validation

The first line of defense in this project is input validation. Before accepting user data, the program verifies email format, password strength, and suggested username restrictions. Rejecting erroneous or unexpected values as soon as feasible is the aim. This adheres to the idea that programs should limit data by type, length, format, and permitted character sets rather than relying on user input [1], [5].

4.2 SQL injection prevention

When untrusted input is processed as a component of a SQL command, SQL injection occurs. There are two ways in which this initiative lowers this danger. First, validation rejects weak inputs and incorrect email values. Second, TypeORM repository techniques are used for database access instead of manual SQL string concatenation. As fundamental SQL injection countermeasures, OWASP suggests parameterized queries and secure data access methods [2].

4.3 Authentication with JWT and cookies

The server generates a JSON Web Token with claims such the user ID, email address, and role following a successful login. JWT is a condensed format used to transmit claims between parties [3]. The JWT is kept in a signed HttpOnly cookie called token in our project. While the signed cookie technique helps identify cookie tampering, HttpOnly helps stop client-side JavaScript from directly accessing the token.

4.4 Safe response objects

Sensitive data, like passwordHash, may be included in a database user entity. This whole object cannot be returned to the client by the API. Rather, the project returns just id, email, username, and role using a mapping function like toSafeUser. This illustrates the idea of reducing data exposure.

5. System Design and Implementation Overview

The implemented backend is a Fastify API that connects to a MariaDB database using TypeORM. The API exposes routes for creating a user, logging in, logging out, and retrieving users. Swagger/OpenAPI is used to document the expected request and response structure. Fastify route schemas are useful because Fastify supports schema-based validation and serialization [4].

Component	Role in the project
Fastify	HTTP server and route handling.
TypeScript	Static typing and clearer backend structure.
TypeORM	Database access through repository methods.
MariaDB	Relational database for storing users.
bcrypt	Password hashing before storage.
JWT	Authentication token containing user claims.

@fastify/cookie	Cookie parsing, signing, setting, and clearing.
@fastify/cors	Allows frontend origins to call the API with cookies during development.
Swagger/OpenAPI	API documentation and manual security testing interface.

6. API Endpoints Used in the Demonstration

Endpoint	Purpose	Security relevance
POST /user	Create a new user account.	Tests input validation, password rules, duplicate email handling, and safe response output.
POST /auth/login	Authenticate a user with email and password.	Tests password verification, JWT creation, and cookie setting.
GET /users	Return all users.	Protected route; tests authentication and safe response mapping.
POST /auth/logout	Clear the authentication cookie.	Tests session termination and access removal.
GET /auth/me	Take the account information	Test if the user can now fetch their own data or not.
GET /users/{id}	Check the user depend on their id	Test if the id is unique and can see the correct information of user or not.

7. Key Implementation Details

7.1 User creation and validation

The user creation route defines the required fields email, username, and password. The email must follow email format rules, and the password must satisfy a strength pattern. For a stronger implementation, username should also be constrained to a safe pattern such as letters, numbers, underscore, and hyphen only.

```

import Joi from "joi";
// This file defines validation schemas for user-related operations using the Joi library. T
export const passwordSchema = Joi.string()
  .min(12)
  .max(128)
  .pattern(/[a-z]/, "lowercase")
  .pattern(/[A-Z]/, "uppercase")
  .pattern(/[0-9]/, "number")
  .pattern(/[^a-zA-Z0-9]/, "special character")
  .required()
  .messages({
    "string.empty": "Password is required",
    "string.min": "Password must be at least 12 characters",
    "string.max": "Password must be at most 128 characters",
    "string.pattern.base":
      "Password must include uppercase, lowercase, number, and special character",
  });

// This file defines validation schemas for user-related operations using the Joi library. T
export const createUserSchema = Joi.object({
  email: Joi.string().email().required().max(255),
  username: Joi.string().min(3).max(50).required(),
  password: passwordSchema,
});

// This file defines validation schemas for user-related operations using the Joi library. T
export const loginSchema = Joi.object({
  email: Joi.string().email().required(),
  password: Joi.string().required(),
});
export const userIdSchema = Joi.object({
  id: Joi.string().uuid().required(),
});

```

7.2 Login and cookie-based JWT authentication

The login route first validates the request body. It then finds the user by email, compares the submitted password with the stored bcrypt password hash, creates a JWT payload, and sets the token in a cookie. If the user does not exist or the password is wrong, the API returns the same message, Invalid credentials, to avoid revealing which part of the login attempt was incorrect.

```

const user = await userRepo.findOne({
  where: { email: body.email },
});

if (!user) {
  return reply.status(401).send({
    message: "Invalid credentials",
  });
}
const passwordOk = await bcrypt.compare(body.password, user.passwordHash);

if (!passwordOk) {
  return reply.status(401).send({
    message: "Invalid credentials",
  });
}

if (!process.env.JWT_SECRET) {
  throw new Error("Missing environment variable: JWTSECRET")
}

const payload: JwtPayload = {
  sub: user.id,
  email: user.email,
  role: user.role,
};

const token = jwt.sign(payload, process.env.JWT_SECRET, {
  algorithm: "HS256",
  expiresIn: "1h",
});
reply.setCookie(authCookieName, token, authCookieOptions);
return reply.send({
  message: "Login successful",
  user: toSafeUser(user),
});
});

```

7.3 Protected route: GET /users

The route GET /users is protected using preHandler: app.authenticate. This means the authentication function runs before the database query. If the request does not contain a valid cookie and JWT, the handler does not run and the server returns 401 Unauthorized.

```

app.get("/users",
{
  schema: {
    tags: ["Users"],
    summary: "Get all users that can be seen depending on role",
    description:
      "Protected route. Try once without login to see 401, then login and try again.",
    security: [{ cookieAuth: [] }],
    response: {
      200: {
        type: "array",
        items: {
          type: "object",
          properties: {
            id: { type: "string" },
            email: { type: "string" },
            username: { type: "string" },
            role: { type: "string" },
          },
        },
      },
      401: {
        type: "object",
        properties: {
          message: { type: "string" },
        },
      },
    },
  },
  preHandler: app.authenticate,
}, async (request, reply) => {
  const users = await userRepo.find();
  return reply.send(users.map(toSafeUser));
});

```

It is important to understand that security: [{ cookieAuth: [] }] in the Swagger schema only documents the requirement. The actual enforcement happens in preHandler: app.authenticate.

8. Swagger-Based Security Testing

Swagger is used as a repeatable test interface. Each route contains examples and expected responses. The main demonstration is based on comparing expected and actual behavior for both normal and malicious inputs.

Test case	Input / action	Expected result	Security meaning
Valid user creation	Create user with valid email and strong password.	201 Created.	The normal business flow works.
Invalid email	Use email value invalid-email.	400 Bad Request.	Malformed input is rejected.

SQL injection-looking email	Use email value ' OR 1=1 -	400 Bad Request.	Payload is blocked by email validation.
Weak password	Use password 123.	400 Bad Request.	Weak credentials are rejected.
XSS-looking username	Use username <code><script>alert(1)</script></code> .	400 Bad Request if username pattern is enabled.	Dangerous display payloads are rejected.
GET /users before login	Call protected route without cookie.	401 Unauthorized.	Authentication is enforced.
GET /users after login	Login, then call GET /users.	200 OK with safe user list.	Cookie authentication works.
Logout test	Logout, then call GET /users again.	401 Unauthorized.	Session termination works.
Duplicate email	Create a user with an already used email.	409 Conflict.	Duplicate account creation is prevented.

9. Results and Security Analysis

9.1 Validation behavior

The validation layer rejects invalid email addresses and weak passwords before the data is stored. This is visible in Swagger as a 400 Bad Request response. For example, a payload such as an email value of ' OR 1=1 -- should not be accepted as a valid email. This shows that basic injection-looking input is blocked at the boundary of the application.

9.2 Authentication behavior

When the protected route GET /users is called prior to login, it returns 401 Unauthorized. The browser obtains the authentication cookie upon a successful login, and the same route returns 200 OK. The cookie is deleted upon logout, and the route returns 401 Unauthorized once more. This indicates that the authentication lifecycle functions as it should.

9.3 Data exposure behavior

Rather of returning whole database instances, the API delivers safe user objects. This indicates that answers do not reveal passwordHash. This is crucial since clients shouldn't be given access to hashed passwords.

9.4 SQL injection evaluation

A SQL injection-looking string can appear in a request, but that does not automatically mean the injection succeeded. Injection would be considered successful if it bypassed login, returned unauthorized database records, caused unintended database modification, or produced SQL errors caused by string concatenation. In this project, the main checks are: the payload is rejected when it violates validation rules; login bypass with ' OR 1=1 -- fails; the database does not contain unexpected records; and TypeORM repository methods treat values as data rather than executable SQL.

10. Discussion and Limitations

Although the project is purposefully small, it illustrates key secure programming concepts. Currently, all users are returned via the GET /users route upon authentication. This route should typically only be available to administrators in a genuine application. To obtain only their own profile, regular users should use a different route, like GET /auth/me.

Additionally, the current version mostly utilizes Swagger for manual testing. Integration tests should be used to automate these tests for a production-quality system. Role-based authentication, refresh token design, more stringent cookie setup for HTTPS deployment, database least privilege, centralized error handling, audit logging, and more thorough test coverage are further enhancements.

11. Conclusion

This project demonstrates secure programming in an understandable and useful way using a basic CRUD API. Input validation, password hashing, JWT-based authentication, signed HttpOnly cookies, secure response mapping, and protected routes are all used in the implementation. Swagger serves as a controlled manual testing environment in addition to being used for documentation. The most significant outcome is that the API may exhibit both secure failure behavior and successful behavior: although invalid input, unauthenticated access, weak passwords, and suspicious payloads are rejected or handled properly, genuine users can be established and authenticated.

The study provides a response to the research question by demonstrating how safe programming concepts may be successfully shown in a small project when the implementation is linked to observable, repeatable test cases and each control's security function is made obvious.

References

- [1] OWASP Cheat Sheet Series. Input Validation Cheat Sheet.
https://cheatsheetseries.owasp.org/cheatsheets/Input_Validation_Cheat_Sheet.html
- [2] OWASP Cheat Sheet Series. SQL Injection Prevention Cheat Sheet.
https://cheatsheetseries.owasp.org/cheatsheets/SQL_Injection_Prevention_Cheat_Sheet.html
- [3] IETF. RFC 7519: JSON Web Token (JWT). <https://datatracker.ietf.org/doc/html/rfc7519>
- [4] Fastify Documentation. Validation and Serialization.
<https://fastify.io/docs/latest/Reference/Validation-and-Serialization/>
- [5] OWASP Cheat Sheet Series. REST Security Cheat Sheet.
https://cheatsheetseries.owasp.org/cheatsheets/REST_Security_Cheat_Sheet.html

Appendix A: Demo Checklist

- Start MariaDB Docker container and backend server.
- Open Swagger UI at <http://localhost:8081/documentation>.
- Create one valid user through POST /user.
- Try invalid email, weak password, and suspicious payload examples.
- Call GET /users before login and confirm 401 Unauthorized.
- Login through POST /auth/login and confirm Login successful.
- Call GET /users again and confirm safe user list.
- Confirm response does not include passwordHash.
- Logout through POST /auth/logout.
- Call GET /users again and confirm 401 Unauthorized.